

Конвейеры обработки данных с TPL Dataflow

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Антон Орехов

программист интеграции

- ≈20 лет опыта в качестве full stack, backend разработчика (C#, Python, SQL)

 @Antoni_www

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **#канал**
группы



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Почему TPL Dataflow?

Основные строительные блоки

Построение конвейера

Управление параллелизмом

Практика

Основные тезисы



Цели вебинара

К концу занятия вы сможете

1. Проектировать и реализовывать сложные конвейеры обработки данных (ETL, workflow);
2. Выбирать подходящие блоки TPL Dataflow для решения конкретной задачи (трансформация, буферизация, агрегация);
3. Управлять степенью параллелизма и противодавлением (back-pressure) в асинхронных потоках данных.

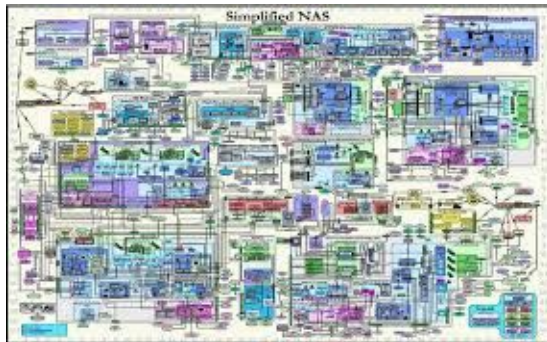


Почему TPL Dataflow?

Асинхронность и параллелизм

Проблемы

- Сложность координации асинхронных задач.
- Управление общим состоянием и гонки данных.
- Противодействие (Back-pressure): как не перегрузить систему?
- "Спагетти-код": тяжело читать, тяжело отлаживать.
- Неэффективное использование ресурсов (CPU, память).

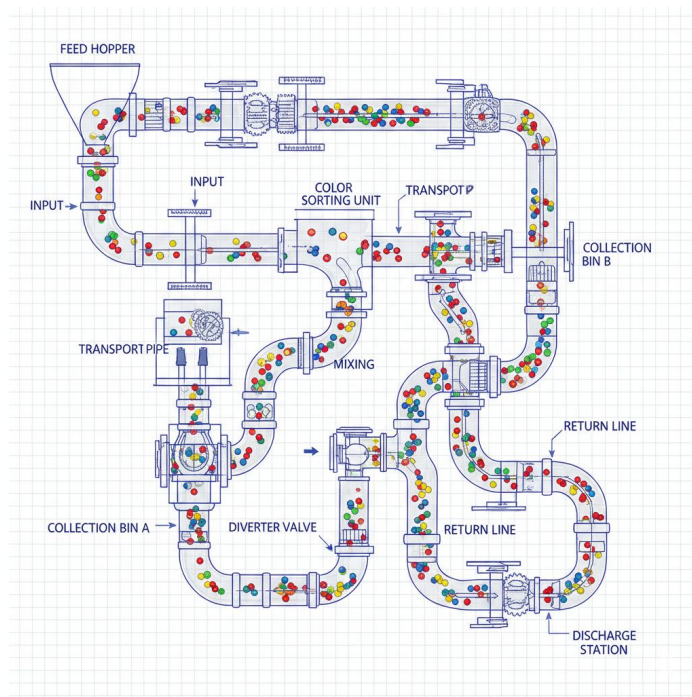


Dataflow и Агенты

- Dataflow (Потоки данных): Данные перемещаются по заранее определенному конвейеру.
- Агенты (Блоки): Независимые, специализированные компоненты, обрабатывающие данные.

Преимущества:

- Четкое разделение ответственности.
- Минимизация общего состояния.
- Встроенное управление потоком данных (включая противодействие).
- Визуально понятная архитектура.



TPL Dataflow — это официальная библиотека от Microsoft, входящая в пространство имён `System.Threading.Tasks.Dataflow`. Она предоставляет нам набор готовых, 'кирпичиков' или 'станков' — блоков, которые мы можем легко связывать между собой для построения сложных конвейеров обработки данных.



TPL Dataflow

- Библиотека System.Threading.Tasks.Dataflow.
- Предоставляет набор готовых блоков для создания конвейеров.
- Основана на TPL (Task Parallel Library).
- Идеальна для задач, где есть поток данных, требующий последовательной обработки этапами.



Где пригождается

- ETL (Extract, Transform, Load): Выгрузка данных, их преобразование, загрузка.
- Парсинг логов/файлов: Чтение, фильтрация, обработка.
- Обработка изображений/видео: Последовательные фильтры или преобразования.
- Workflow-системы: Прохождение документа по стадиям.
- Конвейеры обработки событий/сообщений.



TPL Dataflow vs. TPL/Async-Await

Аспект	TPL Dataflow	TPL / Async-Await
Фокус	Поток данных, конвейеры, агенты	Отдельные асинхронные операции, задачи
Координация	Встроенная (LinkTo, Completion, противодействие)	Ручная (Task.WhenAll, SemaphoreSlim, lock)
Состояние	Минимизировано (блоки независимы)	Чаще всего общее, требует синхронизации
Противодействие	Встроено (BoundedCapacity)	Требует ручной реализации
Сложность для конвейеров	Низкая (высокий уровень абстракции)	Высокая (много ручной логики)
Применимость	Потоковая обработка, конвейеры, workflow	Единичные асинхронные операции, параллельный LINQ



TPL Dataflow

1. Упрощает построение сложных асинхронных конвейеров.
2. Позволяет сфокусироваться на логике обработки, а не на координации.
3. Встроенное управление ресурсами и противодавлением.
4. Делает код более читаемым и поддерживаемым.



ПРИМЕР

Основные строительные блоки TPL Dataflow

Блоки

В TPL Dataflow блоки можно разделить на три основные категории

- **Блоки источника (Source Blocks):** Могут отправлять данные другим блокам.
- **Блоки назначения (Target Blocks):** Могут принимать данные от других блоков.
- **Блоки-агенты (Propagator Blocks):** Могут как принимать, так и отправлять данные.

Базовые блоки буферизации



BufferBlock<T>, BroadcastBlock<T>, WriteOnceBlock<T>

- **Общие параметры:** DataflowBlockOptions
 - CancellationToken: Токен отмены для отмены операций блока.
 - MaxMessages: Максимальное количество сообщений, которое может обработать блок.
 - NameFormat: Формат имени для отладки.



BufferBlock<T>

- **Назначение:** Простой буфер, который хранит элементы до тех пор, пока их не заберут. Может выступать в роли "очереди" или "обменника" данных между блоками.
- **Когда использовать:** Когда вам нужно временно хранить данные, пока блок назначения не будет готов их обработать.
- **Пример:** Представьте, что вы скачиваете файлы из интернета. BufferBlock может хранить URL-адреса, ожидающие обработки, или сами скачанные файлы, ожидающие сохранения.

```
// Создание
var bufferBlock = new BufferBlock<T>();

// Отправка
bufferBlock.Post(item);

// Получение (асинхронно)
T item = await bufferBlock.ReceiveAsync();

// Попытка получения (неблокирующая)
if (bufferBlock.TryReceive(out T item)) { /* ... */ }
```



ПРИМЕР

BroadcastBlock<T>

- **Назначение:** Распространяет каждое полученное сообщение всем связанным блокам назначения.
- **Когда использовать:** Когда одно и то же сообщение должно быть обработано несколькими независимыми потребителями.
- **Пример:** Уведомление нескольких систем о новом заказе или событии.

```
var broadcast = new BroadcastBlock<string>(s => s); // Функция для клонирования, если T - изменяемый тип
var target1 = new BufferBlock<string>();
var target2 = new BufferBlock<string>();

broadcast.LinkTo(target1);
broadcast.LinkTo(target2);

broadcast.Post("Привет, мир!");

string msg1 = await target1.ReceiveAsync(); // msg1 = "Привет, мир!"
string msg2 = await target2.ReceiveAsync(); // msg2 = "Привет, мир!"
```

WriteOnceBlock<T>

- **Назначение:** Аналогично `BufferBlock`, но может принимать только одно сообщение. Все последующие сообщения игнорируются.
- **Когда использовать:** Для кеширования первого вычисленного значения или обеспечения выполнения задачи только один раз.
- **Пример:** Вычисление значения, которое должно быть сделано только один раз, например, начальная конфигурация.

```
var writeOnce = new WriteOnceBlock<int>();  
writeOnce.Post(10);  
writeOnce.Post(20); // Игнорируется  
  
int item = await writeOnce.ReceiveAsync(); // item = 10
```

Блоки выполнения операций



ActionBlock<TInput>, TransformBlock<TInput,TOutput>, TransformManyBlock<TInput,TOutput>

- **Общие параметры:** ExecutionDataflowBlockOptions
 - **MaxDegreeOfParallelism:** Максимальное количество задач, которые могут выполняться параллельно. (По умолчанию = 1, DataflowBlockOptions.Unbounded для неограниченного).
 - **BoundedCapacity:** Максимальное количество сообщений, которые блок может буферизовать перед тем, как перестать принимать новые сообщения.
 - **MaxMessagesPerTask:** Максимальное количество сообщений, которые может обработать одна задача.
 - **TaskScheduler:** Планировщик задач, который будет использоваться для выполнения задач блока.

ActionBlock<T>

- **Назначение:** Выполняет асинхронное действие для каждого полученного элемента. Это блок назначения, который не отправляет данные дальше.
- **Когда использовать:** Для выполнения побочных эффектов, таких как запись в базу данных, отправка электронных писем, вызов внешних API.
- **Пример:** Запись логов, сохранение обработанных данных, отправка уведомлений.



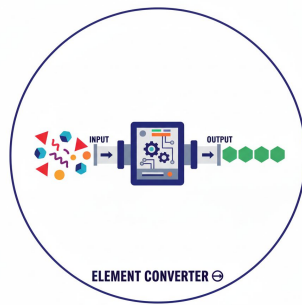
ПРИМЕР

TransformBlock<TIn, TOut>

- **Назначение:** Преобразует данные одного типа (TIn) в данные другого типа (TOut). Это блок-агент.
- **Когда использовать:** Когда вам нужно преобразовать данные из одного формата в другой.
- **Пример:** Парсинг строк в объекты, преобразование типов данных, изменение структуры объекта.

```
// Создание
var transformBlock = new TransformBlock<TIn, TOut>(input => {
    // Логика трансформации
    return transformedOutput;
});

// Использование
transformBlock.Post(inputData);
TOut output = await transformBlock.ReceiveAsync(); // Получаем результат
```



TransformManyBlock<TInput, TOutput>

- **Назначение:** Принимает один элемент TInput и может выводить ноль, один или несколько элементов TOutput. Это блок-агент.
- **Когда использовать:** Когда один входной элемент может породить множество выходных элементов.
- **Пример:** Разделение текстового документа на отдельные слова, декомпрессия архива в список файлов, парсинг списка элементов из JSON-строки.

TransformManyBlock<TInput, TOutput>

```
var transformManyBlock = new TransformManyBlock<string, char>(async input =>
{
    Console.WriteLine($"Разбиение строки '{input}' на символы.");
    await Task.Delay(50);
    return input.ToCharArray();
},
new ExecutionDataflowBlockOptions { MaxDegreeOfParallelism = 1 });

var buffer = new BufferBlock<char>();
transformManyBlock.LinkTo(buffer);

transformManyBlock.Post("ABC");
transformManyBlock.Post("DEF");

char c1 = await buffer.ReceiveAsync(); // c1 = 'A'
char c2 = await buffer.ReceiveAsync(); // c2 = 'B'
// ...
```

Блоки группировки



BatchBlock<T>, JoinBlock<T1,T2>, BatchedJoinBlock<T1,T2>

- **Общие параметры:** GroupingDataflowBlockOptions
 - **Greedy:** Определяет, будет ли блок жадно принимать элементы или ждать, пока все связанные буферы будут готовы. (true по умолчанию для большинства блоков, false для JoinBlock). JoinBlock по умолчанию не жадный, то есть он будет ждать, пока все источники не предоставят элементы, прежде чем принять их.
 - **MaxNumberOfGroups:** Максимальное количество групп, которые может создать блок.

BatchBlock<T>

- **Назначение:** Объединяет несколько входных элементов в массив T[] фиксированного размера.
- **Когда использовать:** Когда вы хотите обрабатывать данные пакетами, например, для оптимизации операций ввода-вывода или пакетной обработки в базе данных.
- **Пример:** Сбор 100 строк логов перед их записью в файл, или отправка 50 записей в базу данных одной транзакцией.

BatchBlock<T>

```
var batchBlock = new BatchBlock<int>(3); // Будет собирать по 3 элемента
```

```
var actionBlock = new ActionBlock<int[]>(batch =>  
{  
    Console.WriteLine($"Обработан пакет: {string.Join(", ", batch)}");  
});  
batchBlock.LinkTo(actionBlock);
```

```
batchBlock.Post(1);  
batchBlock.Post(2);  
batchBlock.Post(3); // Сработает  
batchBlock.Post(4);  
batchBlock.Post(5);  
batchBlock.Post(6); // Сработает
```

JoinBlock<T1, T2>(и перегрузки до T8)

- **Назначение:** Объединяет элементы из нескольких входных буферов, ожидая элемента от каждого буфера перед выпуском кортежа (Tuple<T1, T2>).
- **Когда использовать:** Когда вам нужны данные из нескольких источников для выполнения одной операции.
- **Пример:** Обработка заказа, который требует информации как о клиенте, так и о товаре.

JoinBlock<T1, T2>

```
var joinBlock = new JoinBlock<string, int>();
```

```
var source1 = new BufferBlock<string>();
```

```
var source2 = new BufferBlock<int>();
```

```
joinBlock.Target1.LinkTo(source1); // Связываем входные таргеты JoinBlock с Source
```

```
joinBlock.Target2.LinkTo(source2);
```

```
var actionBlock = new ActionBlock<Tuple<string, int>>(data =>
```

```
{
```

```
    Console.WriteLine($"Объединенные данные: Строка='{data.Item1}', Число={data.Item2}");
```

```
});
```

```
joinBlock.LinkTo(actionBlock);
```

```
source1.Post("Пользователь");
```

```
source2.Post(101); // Сработает, когда оба источника будут иметь по элементу
```

```
source1.Post("Продукт");
```

```
source2.Post(202); // Сработает
```



BatchedJoinBlock<T1, T2>(и перегрузки до T8)

- **Назначение:** Комбинирует функциональность BatchBlock и JoinBlock. Он объединяет пакеты элементов из нескольких входных буферов.
- **Когда использовать:** Когда вы хотите объединить несколько потоков данных в пакеты.
- **Пример:** Объединение пакетов пользовательских действий с пакетами информации о продуктах для аналитической обработки.

BatchedJoinBlock<T1, T2>

```
var batchedJoinBlock = new BatchedJoinBlock<string, int>(2); // Объединяет по 2 элемента из каждого источника
```

```
var sourceStrings = new BufferBlock<string>();
```

```
var sourceInts = new BufferBlock<int>();
```

```
batchedJoinBlock.Target1.LinkTo(sourceStrings);
```

```
batchedJoinBlock.Target2.LinkTo(sourceInts);
```

```
var actionBlock = new ActionBlock<Tuple<IList<string>, IList<int>>>>(data =>
```

```
{
```

```
    Console.WriteLine($"Пакет строк: {string.Join(", ", data.Item1)}");
```

```
    Console.WriteLine($"Пакет чисел: {string.Join(", ", data.Item2)}");
```

```
});
```

```
batchedJoinBlock.LinkTo(actionBlock);
```

```
sourceStrings.Post("A");
```

```
sourceInts.Post(1);
```

```
sourceStrings.Post("B");
```

```
sourceInts.Post(2); // Сработает, когда оба источника получат по 2 элемента
```



ПРИМЕР

Ключевые тезисы

- `ActionBlock<T>`: Действие без вывода.
- `TransformBlock<TIn, TOut>`: Преобразование входного типа в выходной.
- `BufferBlock<T>`: Просто буферизация/хранение данных.
- Все блоки поддерживают асинхронную работу (`Func<T, Task>`, `Func<TIn, Task<TOut>>`).
- Методы `Post()`, `Complete()`, `Completion` - общие для управления.



Вопросы для проверки

1. Какие ключевые опции доступны в `ExecutionDataflowBlockOptions`? Объясните значение `MaxDegreeOfParallelism` и `BoundedCapacity`.
2. Какова основная опция в `GroupingDataflowBlockOptions`? Что означает опция `Greedy` и как она влияет на `JoinBlock`?



Построение конвейера



LinkTo

- **Назначение:** Связывает исходный блок (источник) с целевым блоком (потребителем), направляя выходные данные первого на вход второго.
- **Принцип работы:** Когда исходный блок производит данные, они автоматически отправляются в связанный целевой блок.



```
sourceBlock.LinkTo(targetBlock);  
// Или с опциями  
sourceBlock.LinkTo(targetBlock, new DataflowLinkOptions { PropagateCompletion = true });
```

DataflowLinkOptions

- **PropagateCompletion:** Если установлено в true (по умолчанию), то когда блок-источник завершает свою работу (т.е., вызовут Complete() на нем), он также вызовет Complete() на всех связанных блоках-назначениях. Это позволяет каскадно завершать весь конвейер.
- **MaxMessages:** Указывает максимальное количество сообщений, которое может быть передано по данной связи. После передачи этого количества сообщений связь будет разорвана.
- **Append:** Если true (по умолчанию), связь добавляется в конец списка связей блока-источника. Если false, связь добавляется в начало списка. Это влияет на порядок, в котором сообщения будут предлагаться связанным блокам, если их несколько.
- **Predicate:** Делегат `Func<TOutput, bool>`, который действует как фильтр. Только те сообщения, для которых предикат возвращает true, будут отправлены по этой связи. Это позволяет маршрутизировать сообщения в зависимости от их содержимого.



Простой конвейер TransformBlock -> ActionBlock

Задача:

- Принять список строк.
- Преобразовать каждую строку в её длину (int) с помощью TransformBlock.
- Вывести полученные числа на консоль с помощью ActionBlock.



ПРИМЕР

Как данные двигаются по конвейеру?

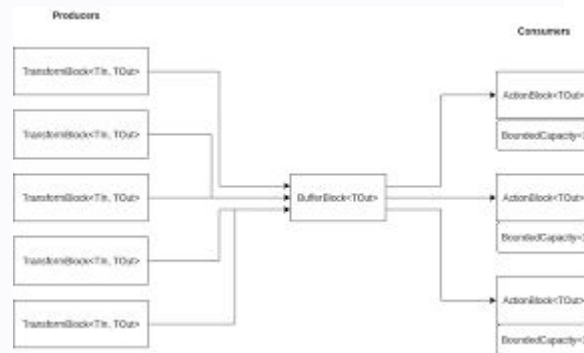
- Непрерывный поток: Данные не ждут завершения всех предыдущих элементов.
- Параллельная обработка: Пока `stringToLengthBlock` обрабатывает один элемент, он уже может передать результат предыдущего элемента в `printNumberBlock`, который начнет его обработку.
- Внутренние буферы: Блоки имеют внутренние буферы (очереди), куда поступают данные. Если блок занят, данные ждут в буфере.
- `Post()` / `SendAsync()`: Отправляет элемент на вход блока. `SendAsync` вернет `Task`, который завершится, когда элемент будет принят.
- `Complete()`: Сигнал о том, что больше данных не будет. Важен для корректного завершения.
- `Completion`: `Task`, который завершается, когда блок обработал все данные и закрылся.

**Управление параллелизмом и
противодавлением (back-pressure)**



ExecutionDataflowBlockOptions

- **Что это:** Класс, который передается в конструктор блоков, чтобы настроить их внутреннее поведение.
- **Ключевые параметры:**
 - `MaxDegreeOfParallelism`: Сколько элементов обрабатывать одновременно.
 - `BoundedCapacity`: Максимальный размер внутреннего буфера блока.
 - `CancellationToken`: Для отмены операций.



MaxDegreeOfParallelism

- **Назначение:** Определяет максимальное количество сообщений, которые блок может обрабатывать одновременно.
- **Значения:**
 - 1: Последовательная обработка (по одному элементу за раз).
 - `DataflowBlockOptions.Unbounded` (или -1): Максимально возможное параллельное выполнение (сколько позволяют ресурсы, обычно пул потоков).
 - N (любое целое число > 0): Фиксированное количество параллельных операций.
- **Важность:** Баланс между использованием ресурсов и отзывчивостью.

ПРИМЕР

BoundedCapacity

- Назначение: Устанавливает максимальное количество элементов, которые могут одновременно находиться в буфере ожидания блока.
- Роль в противодавлении:
 - Когда буфер заполнен, метод `Post()` или `SendAsync()` в предыдущий блок начинает ждать (асинхронно), пока в текущем блоке не освободится место.
 - Это автоматически замедляет 'производителя' данных, предотвращая переполнение 'потребителя'.

ПРИМЕР

Почему это важно?

- **Предотвращение перегрузки:** Защищает систему от исчерпания памяти или CPU, когда производитель данных работает быстрее потребителя.
- **Стабильность системы:** Гарантирует, что медленные компоненты не приведут к сбоям более быстрых.
- **Автоматическое управление:** Не нужно писать сложную логику ручной синхронизации (семафоры, очереди).
- **Эффективное использование ресурсов:** Система работает на максимально возможной скорости без коллапса.



Завершение работы конвейера



Complete()

- **Назначение:** Сигнализирует блоку (или всему конвейеру), что больше никаких входных сообщений не будет.
- **Действие:** Блок перестает принимать новые сообщения, но продолжает обрабатывать те, что уже есть в его буфере.
- **Цель:** Инициировать процесс завершения.



Completion (свойство, возвращает Task)

- **Назначение:** Task, который завершается, когда блок (или весь конвейер) полностью обработал все свои сообщения, освободил все ресурсы и закрылся.
- **Использование:** `await block.Completion;` - позволяет дождаться полного завершения.



Корректное завершение конвейера

- **Задача:** Убедиться, что наш предыдущий конвейер корректно завершается и мы ждем его полного выполнения.
- **Ключевые шаги:**
 - Вызвать `Complete()` на начальном блоке конвейера.
 - Дождаться `Completion` на конечном блоке конвейера (благодаря `PropagateCompletion`).

Базовый механизм обработки ошибок

- Когда исключение возникает в делегате блока (например, ActionBlock или TransformBlock):
 - TPL Dataflow перехватывает исключение.
 - Completion Task блока переходит в состояние Faulted.
 - Последующие сообщения, отправленные в этот блок, будут отклонены.
- **Как отловить:** await block.Completion в try-catch блоке, чтобы получить AggregateException.

```
var failingBlock = new ActionBlock<int>(i => {  
    if (i == 2) throw new Exception("Ошибка!");  
});  
try {  
    await failingBlock.Completion;  
} catch (AggregateException ae) {  
    // Обработка InnerExceptions  
}
```

Проблема с базовым подходом

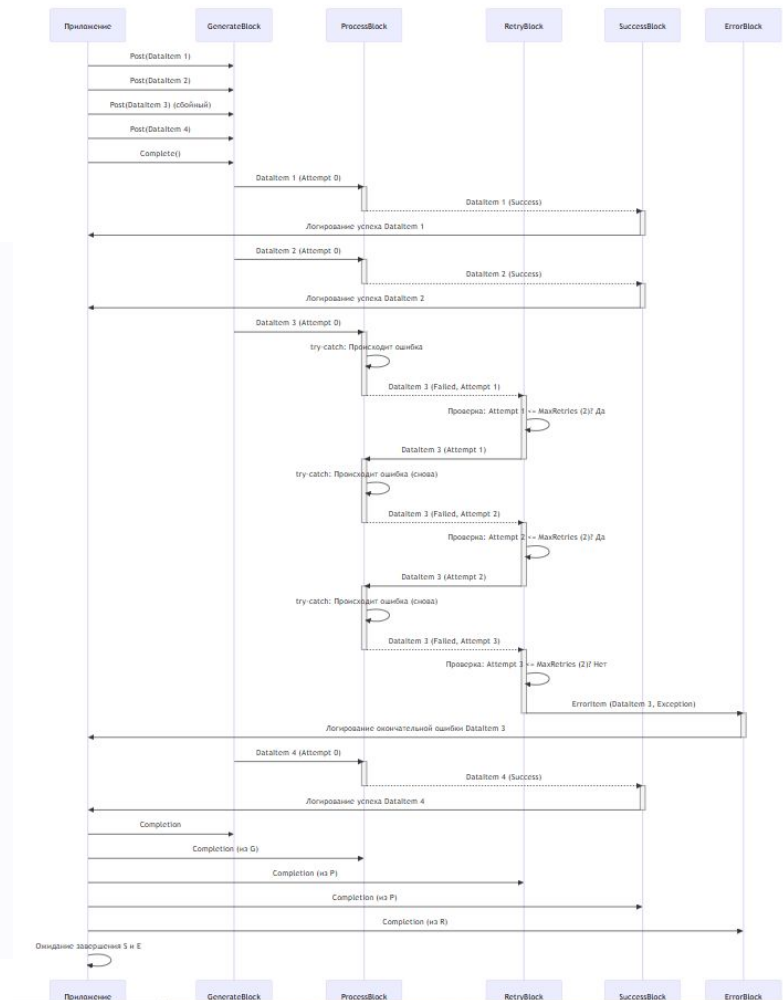
- Полная остановка: Один сбойный элемент может "сломать" весь блок, останавливая обработку всех остальных элементов.
- Потеря контекста: Простое Completion.Exception не дает информации о том, какой именно элемент вызвал сбой.
- Отсутствие гибкости: Невозможно реализовать повторные попытки, логирование конкретной ошибки или перенаправление.



Стратегии надежной обработки ошибок

- "Защитное" программирование в делегате:
 - Оборачивайте логику обработки каждого элемента в try-catch внутри делегата блока.
 - Это позволяет блоку продолжать обрабатывать другие элементы, даже если один из них вызвал ошибку.
- Возврат "статусных" объектов: Пусть блок `TransformBlock<TInput, TOutput>` возвращает объект `Result<T>` или `ProcessedItem` со статусом (Success, Failed) и деталями ошибки, а не выбрасывает исключение.
- Блоки повторных попыток (Retry Blocks): Специальный `ActionBlock`, который получает сбойные элементы, увеличивает счетчик попыток и, если лимит не исчерпан, отправляет их обратно в основной блок.
- Блоки ошибок (Error Blocks / Dead Letter Queues):
 - Специальный `ActionBlock`, который получает элементы, полностью исчерпавшие попытки, или элементы с невозстановимыми ошибками.
 - Отвечает за логирование, оповещения, сохранение для ручного анализа.
- Предикаты `LinkTo()` для маршрутизации: Используйте `linkTo(target, predicate)` для направления успешных и ошибочных элементов в разные ветви конвейера.

- **Приложение (App):** Иницирует отправку данных и ожидает завершения всего конвейера.
- **GenerateBlock (G):** Блок-источник, который генерирует и отправляет элементы данных в конвейер.
- **ProcessBlock (P):** Основной блок обработки. Он включает внутренний механизм try-catch, который позволяет ему обрабатывать ошибки для отдельных элементов без остановки всего блока.
- **RetryBlock (R):** Отвечает за логику повторных попыток. Он получает элементы, которые не удалось обработать, и решает, отправить их обратно в ProcessBlock или переместить в ErrorBlock.
- **SuccessBlock (S):** Блок назначения, который получает и обрабатывает элементы, успешно прошедшие ProcessBlock.
- **ErrorBlock (E):** Блок назначения для элементов, которые не удалось обработать даже после всех повторных попыток. Действует как "Dead Letter Queue".



ПРИМЕР

Вопросы для проверки

1. Какова основная опция в GroupingDataflowBlockOptions? Что означает опция Greedy и как она влияет на JoinBlock?
2. Объясните разницу между Complete() и Completion. Как они используются для управления жизненным циклом конвейера?



Вопросы



если есть вопросы



если вопросов нет

Сценарии, Многопоточность и Практика

Сценарии использования

Задача: Вам нужно создать систему, которая скачивает список URL-адресов, затем парсит HTML-контент каждого адреса, извлекает все ссылки и сохраняет их в базу данных.

- Какие блоки TPL Dataflow вы бы использовали для каждого этапа?
- Как бы вы связали эти блоки?
- Какие опции (например, для параллелизма или буферизации) вы бы рассмотрели?



ПРИМЕР

Сценарии использования

Задача: Вы разрабатываете систему обработки логов. Логи поступают в виде строк, их нужно разобрать, определить тип сообщения (ошибка, предупреждение, информация) и отправить в соответствующие хранилища (например, ошибки в отдельный файл, предупреждения в другой, информацию в консоль).

- Какие блоки вы бы использовали?
- Как бы вы организовали маршрутизацию сообщений на основе их типа?



ПРИМЕР

Сценарии использования

Задача: Вам нужно обрабатывать финансовые транзакции. Каждая транзакция состоит из двух частей: данные о покупателе (CustomerData) и данные о покупке (OrderData), которые поступают из разных систем. Для обработки транзакции нужны обе части. Кроме того, транзакции должны обрабатываться пачками по 10 штук для отправки в платежную систему.

- Какие блоки вы бы использовали для сбора данных о покупателе и покупке?
- Какой блок вы бы использовали для пакетной обработки объединенных данных?
- Как бы вы обеспечили, чтобы транзакции обрабатывались только после получения обеих частей?



ПРИМЕР

Вопросы



если есть вопросы



если вопросов нет

**Подведём
итоги занятия**

TPL Dataflow:

Что такое TPL Dataflow?

- TPL Dataflow (Task Parallel Library Dataflow) – это библиотека, входящая в состав .NET Framework (и .NET Core/5+), которая предоставляет модель акторов и конвейеров для обработки данных. Она базируется на Task Parallel Library (TPL) и предназначена для создания асинхронных конвейеров данных.

Для чего она предназначена?

- Она предназначена для облегчения построения систем, где данные перемещаются между независимыми компонентами (блоками), которые выполняют различные операции обработки асинхронно и параллельно.



TPL Dataflow:

Какие основные проблемы она решает?

- Координация асинхронных операций: Упрощает управление потоком данных между асинхронными задачами, избегая сложных ручных механизмов синхронизации.
- Параллелизм: Позволяет легко настроить степень параллелизма для каждого этапа обработки, оптимизируя использование ресурсов.
- Буферизация: Автоматически управляет буферизацией данных между этапами, предотвращая переполнение или простои.
- Обработка потоковых данных: Идеально подходит для обработки непрерывных потоков данных, таких как события, сообщения или файлы.
- Модульность и композиция: Позволяет строить сложные конвейеры из простых, переиспользуемых блоков, улучшая читаемость и поддерживаемость кода.
- Обработка ошибок: Предоставляет механизмы для распространения ошибок по конвейеру и их централизованной обработки.



TPL Dataflow:

Блоки TPL Dataflow делятся на три основные категории по их способности принимать и/или отправлять данные:

- Блоки источника (Source Blocks): Могут отправлять данные другим блокам, но не могут принимать их. В чистом виде таких блоков нет, но `BufferBlock<T>` или `BroadcastBlock<T>` могут выступать в роли источника, если их просто наполнять данными, а не связывать с другими блоками как назначение. Часто блоки-агенты используются как источники.
- Блоки назначения (Target Blocks): Могут принимать данные от других блоков, но не могут отправлять их дальше. Пример: `ActionBlock<TInput>` (выполняет действие и ничего не возвращает).
- Блоки-агенты (Propagator Blocks): Могут как принимать данные от других блоков, так и отправлять их дальше. Это наиболее распространенный тип, так как они формируют основу конвейера. Пример: `TransformBlock<TInput, TOutput>` (принимает `TInput`, преобразует и отправляет `TOutput`).



Результаты вебинара

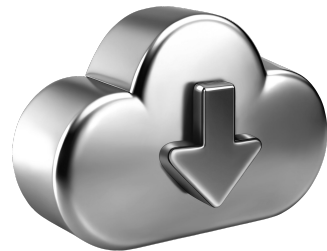
Оцените насколько к концу занятия вы можете:

1. Проектировать и реализовывать сложные конвейеры обработки данных (ETL, workflow);
2. Выбирать подходящие блоки TPL Dataflow для решения конкретной задачи (трансформация, буферизация, агрегация);
3. Управлять степенью параллелизма и противодавлением (back-pressure) в асинхронных потоках данных.



Список материалов для изучения

1. <https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/dataflow-task-parallel-library>
2. <https://www.blinkingcaret.com/2019/05/15/tpl-dataflow-in-net-core-in-depth-part-1/>
3. <https://www.blinkingcaret.com/2019/06/05/tpl-dataflow-in-net-core-in-depth-part-2/>
4. https://www.youtube.com/watch?v=AFMv_nFIvfk





Отправьте в чат эмодзи, который отражает ваше настроение после занятия



Продолжите высказывание: теперь я умею/знаю...



Что планируете использовать в работе?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

